ORTHOGONALITY · LEAST SQUARES

Four ways to compute the same solution

All four minimise $\|A\mathbf{x} - \mathbf{b}\|^2$ and return the same $\hat{\mathbf{x}}$ in exact arithmetic. In floating point they do not, and the difference is entirely about whether the method forms $A^\mathsf{T}A$ or avoids it.

4
METHODS

FORMS $A^\mathsf{T}A$ — CORRECT, FRAGILE — 1

1 **Normal equations**

$\sim mn^2 + n^3/3$; needs full column rank

$$A^\mathsf{T}A\hat{\mathbf{x}} = A^\mathsf{T}\mathbf{b}$$

The derivation everyone learns and the computation nobody should run. Setting the gradient to zero gives these equations directly, and $A^\mathsf{T}A$ is symmetric positive definite so Cholesky solves it — but the squared condition number means the answer can lose half its digits before Cholesky begins.

AVOIDS IT — THE PRACTICAL ROUTES — 3

2 **QR factorization**

$\sim 2mn^2$ via Householder

$$R\hat{\mathbf{x}} = Q^\mathsf{T}\mathbf{b}$$

Factor $A = QR$, multiply through by Q^T, and back-substitute on the triangular system. Q is orthogonal so it preserves lengths and therefore the condition number — the workhorse in every numerical library, and the reason QR is taught alongside least squares.

3 **Pseudoinverse via SVD**

no rank condition at all

$$\hat{\mathbf{x}} = V\Sigma^{+}U^\mathsf{T}\mathbf{b}$$

The only method that works when A is rank-deficient, where the others have no unique solution to find. Among the infinitely many minimisers it returns the one of smallest norm — a choice made deliberately rather than by accident of the algorithm. Costs more than QR and is worth it when the rank is in doubt.

4 **Iterative methods**

one matrix-vector product per step

Krylov iteration on $A^\mathsf{T}A\hat{\mathbf{x}} = A^\mathsf{T}\mathbf{b}$

Never forms $A^\mathsf{T}A$ explicitly — only its action on a vector, which costs two multiplications by A. The right choice when A is large and sparse and a factorization would destroy the sparsity. Convergence depends on conditioning, so preconditioning is part of the method rather than an optimisation.

Forming $A^\mathsf{T}A$ squares the condition number — a matrix with $\kappa = 10^4$ becomes 10^8, and roughly half the available digits are lost before the solve begins. Every method below except the first is a way of not doing that, which is why the textbook derivation and the library implementation take different routes to the same answer.